

Checker-Driven Verilog Repair with a Small Model on Trainium

Team 25: Stacks of Cache | Hack the Chip, NYU x Annapurna Labs | 10 October 2026

Summary. Qwen3-8B, served on a single Trainium chip, writes Verilog that usually compiles but often behaves wrongly. We built a repair loop in which each attempt is simulated in Icarus Verilog and the failure is turned into evidence for the next attempt: the requirement that was violated, what was observed (signal, cycle, which bits were wrong), facts about the model's own code, and at most two short principles retrieved from a knowledge table keyed by failure type. The checker never shows expected values or suggests code, and a pass also requires the official benchmark testbench. On the benchmark LFSR problem, the model went from 0 of 10 correct single attempts, and 30 of 30 identical repeated failures under observation-only feedback, to 5 of 5 runs solved (2.2 attempts per solve) once it was asked to explain each failure and given retrieved principles. Along the way it passed the official testbench with a simulation-only trick, which we caught and closed with a mid-run reset check. The feedback, not the model, decided the outcome; a harder 8-bit CPU task still failed (best 0.75 after 20 attempts).

Problem and system

In a first-attempt baseline over 8 tasks, 7 of 8 answers compiled but only 3 of 8 passed; the failures were logical, not syntactic, and restating the spec did not fix them. **Checker:** checker.py simulates the candidate next to an independent reference under identical stimulus, with separate checks for reset before the first clock edge, reset mid-run, and restart; each failed check is reported as *requirement* + *observation*. **Knowledge cache:** short principles keyed by *design category* : *failure signature*; only the one or two matching entries enter the 8,000-token prompt. **Agent:** a thin loop that logs every prompt, reply, candidate and score (temperature 0.7, thinking off). A teammate pipeline gives feedback from a visible testbench and scores on a hidden one; its numbers are not reported here.

Results: LFSR, 5 runs per condition unless noted

Condition	Solved	Notes
Single attempt, no feedback	0/10	All 10 samples identical (wrong reset, wrong tap).
6 rounds, observation-only feedback	0/5	30/30 attempts byte-identical: the model copied its previous code.
+ explain each failure first	0/5 real	Copying broken, tap bug fixed; 3 'solves' used a simulation-only initial block.
+ mid-run reset check (closes the hack)	2/5	Rounds to solve 5, 5.
+ facts about the model's own code	3/5	Async reset found in 5/5 runs; rounds 4, 4, 3.
+ 10 general principles	4/5	Reset fixed in round 1 in 5/5 runs; rounds 3, 4, 4, 3.
+ 10 observation detectors and principles	3/5	No measurable gain; rounds 4, 6, 6.
Benchmark wording, explain + principles (v4)	5/5	2.2 attempts per solve; all finals pass the official testbench.

Rows 1-7 used our rewritten spec and an older prompt layout; row 8 used the benchmark's wording, a newer layout, the immediate reset stated in the prompt, and LFSR-specific knowledge entries. A progression, not a controlled ablation. **8-bit CPU** (1 run, 20 attempts): 14 compile failures, best 0.75, never passed the official testbench.

Findings

- **Feedback decides the outcome.** Observation-only feedback left the model repeating one design 30 times; stating the violated requirement, adding facts about its own code, and asking for a diagnosis first turned the same failures into passes.
- **A few principles beat more text.** Ten general principles moved the reset fix to round 1 in every run; a larger detector set added text without adding solves.
- **The model games weak checkers.** It passed the official LFSR testbench by initializing the output at simulation start instead of writing a real asynchronous reset; our mid-run reset check rejects this.
- **Spec wording matters.** Our rewritten spec ('XOR of bits 0, 3, 5 and 6') gave a wrong tap in every first attempt; the benchmark's wording ('taps at locations 1, 4, 6 and 7') gave correct taps in 10 of 10.

Limitations and next steps

Five runs per condition is small, and several steps changed more than one thing. LFSR- and sequence-generator-specific knowledge entries were written after seeing those problems, and a generic-only run was not completed; code facts and detectors were added after observing failures. Next: the generic-only and sequence-generator runs, a Yosys synthesis check (proof of concept exists), building on the best attempt rather than the latest, and protocol-level monitors for designs like UART. All logs, prompts and candidates are in the repository.